CASE STUDY / 2026

Karbonomics Impact Engine

A lineage-first proof of concept for clothing Product Carbon Footprint calculation, quality validation, ETL, and reporting.

Python | CSV | SQLite | SQL | ReportLab | Data Quality | Product Carbon Footprint

PREPARED BY

Mudasir Shafiq

Data & AI Engineer | chmudasir.fr@gmail.com

SECTION 00

Contents

A technical walkthrough of the problem, architecture, implementation, evidence, and next stage.

01	**Project Overview**	Page 3
02	**Platform Architecture**	Page 4
03	**Lineage & Data Model**	Page 5
04	**The Engineering Challenge**	Page 6
05	**Implementation Approach**	Page 7
06	**Quality & Verification**	Page 8
07	**Outcome & Next Stage**	Page 9

Evidence standard

All carbon factors and source records in this proof of concept are synthetic and illustrative. The case study documents the implemented engineering method, validation evidence, and reporting flow - not a scientifically validated footprint claim.

SECTION 01

Project Overview

A transparent data path from product inputs and material mappings to reproducible carbon contributions and a readable impact report.

Karbonomics Impact POC demonstrates how a clothing product can move through Core Inputs, material mapping, process parameters, environmental factors, calculation, database loading, quality validation, and PDF reporting without losing lineage.

The example models one cotton T-shirt and uses synthetic factors so the engineering can be tested independently of commercial or scientific data licensing. Every contribution retains source IDs, mapping and factor versions, units, quantities, and the exact arithmetic that produced the result.

3.186	7	17/17	16
kgCO2e illustrative total	traceable contributions	quality checks passed	automated tests

Lineage before totals

Every result can be traced from the product input through canonical material, process, parameter, factor, version, and contribution.

Fail closed

Missing IDs, factors, mappings, incompatible units, or invalid quantities stop calculation before any result is written.

Reproducible ETL

CSV sources load into relational tables with primary keys, foreign keys, checks, and rerun-safe replacement behavior.

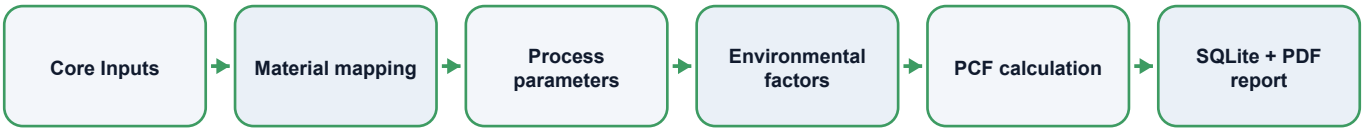
Honest scope

Every output identifies factors as illustrative and avoids presenting the POC as an official carbon methodology.

SECTION 02

Platform Architecture

A small but complete analytical chain separates raw inputs, semantic mappings, process parameters, factors, calculations, quality results, and reporting.



Layer	Implemented responsibility
CI - Core Inputs	Defines the clothing product, material quantities, manufacturing activities, units, and functional unit.
KM - Materials	Maps source material descriptions to canonical material entities and records accepted, pending, or rejected mapping status.
KP - Processes	Connects materials and activities to processes and typed parameters such as kilograms, kWh, or litres.
Factors	Links process parameters to versioned illustrative emission factors with explicit input and output units.
Results	Calculates contributions, product totals, data-quality outcomes, relational tables, and a seven-page PCF report.

Calculation rule

For each eligible input, contribution equals quantity multiplied by factor value. Unit compatibility and accepted mappings are verified before multiplication, so arithmetic never hides semantic mismatches.

SECTION 03

Lineage & Data Model

The relational model preserves the semantic path between product, source input, mapping, process, parameter, factor, run, and result.

Product inputs

One product owns material inputs and manufacturing activities, each with positive quantities and declared units.

Semantic mappings

Raw materials and activities connect to canonical materials and process definitions through explicit mapping tables.

Versioned factors

Environmental factors retain identity, input unit, output unit, value, version, source status, and illustrative flag.

Calculation records

Each run stores row-level contributions and a product summary while preserving all upstream IDs and versions.

Entity path	Key contract	Result
Material	CI material -> KM material -> KP process -> parameter -> factor	Three material contribution paths
Activity	CI activity -> KP process -> unit-matched parameter -> factor	Four activity contribution paths
Quality	IDs, mappings, factors, units, quantities, statuses, reproducibility	17 recorded PASS checks
Storage	Primary keys, foreign keys, CHECK constraints, idempotent load	Zero foreign-key violations

The database contains product, material, activity, mapping, process, parameter, factor, run, contribution, summary, and quality tables. Foreign-key checks run after loading so an apparently successful import cannot conceal broken lineage.

SECTION 04

The Engineering Challenge

Carbon totals are only useful when mappings, factors, units, versions, and assumptions remain visible and reproducible.

A simple multiplication can produce a plausible number even when the wrong factor, unit, mapping, or process has been selected. The POC therefore treats semantic validation and traceability as first-class outputs alongside the final total.

Mapping ambiguity

Raw material and activity labels must resolve to exactly one accepted canonical path before calculation.

Unit compatibility

Kilograms, kWh, litres, and per-piece quantities cannot share factors unless the declared input unit matches.

Missing factors

A process parameter without a factor would silently understate impact unless calculation fails immediately.

Version drift

Mappings and factors can evolve, so each contribution must retain the versions used in that run.

Reproducibility

The same inputs should always regenerate the same seven contributions and 3.186 kgCO2e illustrative total.

Claim discipline

Synthetic factors must be unmistakable so engineering evidence is never misrepresented as scientific validation.

Scope protection The result proves the data architecture and calculation workflow. It is not suitable for public carbon comparisons, compliance reporting, or product claims until authoritative factors and methodology governance are introduced.

SECTION 05

Implementation Approach

Four Python modules create a deterministic chain from source CSVs to validated relational data and a generated report.

1. Validate sources

Check identifiers, duplicates, accepted mappings, factors, units, positive quantities, allowed statuses, and expected reproduction.

2. Calculate contributions

Resolve material and activity paths, multiply quantity by factor, and write seven row-level lineage records.

3. Summarize product

Aggregate contributions for one functional unit while retaining an illustrative flag and run metadata.

4. Load relational store

Create SQLite tables and constraints, then load sources and outputs with rerun-safe INSERT OR REPLACE semantics.

5. Verify database

Enable foreign keys, run foreign-key checks, inspect row counts, and reproduce the total from persisted contributions.

6. Generate report

Read live CSV/database outputs into a seven-page PDF with methodology, result, breakdown, lineage, quality, and limitations.

No hidden constants

The PDF result, breakdown, quality count, and lineage example are read from generated data. Changing the source inputs and rerunning the pipeline changes the report without hand-editing the document.

- The implementation uses Python standard-library modules for CSV, validation, calculation, SQLite loading, and tests; ReportLab handles document generation.
- Input CSV files remain read-only; generated contributions, summaries, database, quality report, and PDF live under the output directory.
- Database constraints duplicate critical validation rules so malformed data is rejected even outside the Python pipeline.

SECTION 06

Quality & Verification

The POC combines pre-calculation checks, automated unit tests, relational integrity checks, and visual report verification.

Evidence	Verified result	Why it matters
Data quality	17 PASS, 0 FAIL	Duplicates, mappings, factors, units, quantities, statuses, and reproducibility are recorded.
Quality tests	9 tests	Known-bad duplicates, mappings, factors, units, quantities, and statuses are rejected.
Database tests	7 tests	Foreign keys, idempotent reloads, constraints, and persisted total reproduction are verified.
Calculation	7 contributions = 3.186 kgCO2e	The saved total reconciles exactly with the contribution breakdown.
Reporting	7-page generated PCF report	Rendered pages were checked for clipping, overlap, chart, table, and warning visibility.

Current boundary

The product, activity data, mappings, and factors are synthetic. The POC intentionally does not claim alignment with an official company methodology, verified life-cycle assessment standard, or production factor database.

- Source files are never mutated; validation and calculation write only to output artifacts.
- Every row retains source IDs and versions so no contribution becomes an unexplained aggregate.
- The report repeats the synthetic/illustrative warning on every page and in the assumptions section.

SECTION 07

Outcome & Next Stage

A compact reference architecture for turning product and process data into traceable impact calculations without sacrificing transparency.

Explainable	Reproducible	Extensible
Every kgCO2e contribution has an inspectable data path.	Validation, ETL, calculation, and report generation rerun deterministically.	Authoritative factors and more products can enter the same contracts.

The POC proves that calculation logic, lineage, data quality, relational integrity, and stakeholder reporting can operate as one system. This creates a safer starting point for introducing governed factors, supplier data, allocation rules, and additional clothing products.

NEXT STAGE

- Replace illustrative factors with licensed, versioned, authoritative sources and formal methodology governance.
- Add supplier-specific data, uncertainty ranges, allocation rules, and scenario comparison.
- Scale from one T-shirt to a governed product portfolio with scheduled ingestion and review workflows.

LET'S TALK

Mudasir Shafiq

Data & AI Engineer | chmudasir.fr@gmail.com

Stack: Python | CSV | SQLite | SQL | ReportLab | Data Quality | Product Carbon Footprint